

- [JARVIS Operationalization Architecture Plan — Room OS Edition](#)
 - [1. Executive Summary](#)
 - [2. Architectural Principles](#)
 - [3. Phase Roadmap \(Phase 10 → Phase 20\)](#)
 - [4. Recommended Software Stack](#)
 - [5. Installation Roadmap](#)
 - [6. Target Runtime Topology](#)
 - [7. Data Flow Architecture](#)
 - [8. Room OS Model](#)
 - [9. Fake Device / Fake Hue Strategy](#)
 - [10. Real Hue / Real Device Integration Strategy](#)
 - [11. Authority Surface Map](#)
 - [12. Governance Invariants to Preserve](#)
 - [13. Iron-Man JARVIS Capability Trajectory](#)
 - [14. Risk Register](#)
 - [15. Testing and Audit Strategy](#)
 - [16. Implementation Style \(Codex Prompt Guidance\)](#)
 - [17. First 15 Codex Slices](#)
 - [18. Move-In Readiness Plan](#)
 - [19. Fortress Integration Summary](#)
 - [20. Final Recommendation](#)

JARVIS Operationalization Architecture Plan — Room OS Edition

Status: Architectural design, pre-implementation **Author:** Claude (senior systems architect role) **Date:** 24 May 2026 **Companion docs:** Architecture v3.1, Phase 1-9 Handoff Report **Scope:** Phases 10-20. Moves JARVIS from governed scaffolds to a real local one-room AI operating system.

1. Executive Summary

The next era of JARVIS turns the governance-first scaffolds from Phases 1-9 into operational software running on a single MacBook, controlling a single bedroom-scale environment, with real local services, real model providers, real UI rendering, and eventually approval-gated execution. This is **Room OS, not whole-home automation**. One room first. One Mac first. Fake devices before real devices. Demo-safe by default.

The strategic shift is from “typed governance contracts exist” to “the same contracts now route real bytes, render real pixels, and gate real side effects”. Nothing in Phases 1-9 gets opened for redesign. Every operationalization slot already exists as a sealed contract — Phase 10+ plugs implementations into those slots without weakening any invariant.

The deliberate sequencing exploits the user’s current advantage (MacBook and room hardware not yet purchased): build the fake room, the demo Command Center, the local-service substrate, the adapter conformance tests, and the bootstrap scripts **first**, so that when the MacBook arrives by early July, the entire system clones, boots, and demos with zero hardware dependency. Real Hue lights, real sensors, and real cameras then plug into existing governed slots one device at a time, behind dry-run gates and approval lifecycle.

The result is a JARVIS that, by end of Phase 20, feels Iron-Man-adjacent (fast conversational UX, visible Command Center, room awareness, voice interaction, visual perception, project memory, replayable reasoning, ambient room control with approval) while remaining inspectable, replayable, and safe.

2. Architectural Principles

Principle	Meaning
One-room-first	First deployment is a single bedroom workspace. Multi-room comes after Room OS is stable for 30 days.
Local-first by default	Cloud is opt-in, budget-gated, consent-gated, audited. Local model routing is the default path.
Fake-first / provider-later	Every provider surface ships with a deterministic mock that passes the same conformance tests. Real provider is one config flag away.
Mock-first / hardware-later	Every hardware adapter ships with a fake adapter that passes the same conformance tests. Real hardware is one config flag away.
Adapter conformance before real hardware	A real Hue light cannot be wired until the fake Hue adapter passes the full conformance suite. The fake is the contract.
No authority without approval	Every side-effecting action enters the approval lifecycle. Voice, vision, scheduler, and observability never bypass it.
No raw payloads in telemetry/UI	Telemetry, replay, Command Center, and Demo Mode use binned/redacted metadata. Raw frames, voice, prompts, outputs, project bodies, secrets remain forbidden.
No hidden capture	Every microphone, camera, screen, or sensor read is visible to the user via active indicator. No background capture.
No always-listening	Push-to-talk only. No wake word in any phase of this plan.
No autonomous execution	The scheduler emits suggestions. The approval lifecycle is the only path to side effects.
No graph-driven execution	Replay graphs, dependency graphs, governance graphs are read-only viewers. They cannot trigger actions.
Read-only observability	The Command Center has no run/retry/approve/mutate affordances. Read-only is structural, not policy.
Deterministic tests and closeout gates	Every phase ships with closeout tests that prove what is disabled. Frozen phases stay frozen. Phases 1-9 are cited, not re-derived. Operationalization plugs into

Frozen phases are substrate, not redesign targets	existing slots.
Real devices plug into governed slots, not added as shortcuts	A real Hue light is a fake light with a different adapter. Not a new code path.

3. Phase Roadmap (Phase 10 → Phase 20)

Phase 10 — Local Bootstrap & Room OS Substrate

Purpose: Make the repo Mac-ready, define the room topology, build the fake room. No UI rendering, no real providers, no real hardware. Pure substrate.

Implemented: - scripts/bootstrap.sh (macOS + Linux): node/pnpm version check, env init, dependency install, test run - scripts/doctor.ts: environment checker (node version, OS, available memory, disk, port availability, missing env vars) - .env.example with every flag set to safe defaults (all real providers disabled) - config/room/demo-room.yaml: default room profile, fake devices, fake sensors - src/room/registry.ts: RoomProfile / Zone / Device / Capability schemas - src/room/adapters/contract.ts: typed adapter interface - src/room/adapters/fake-room-adapter.ts: reference implementation with deterministic state, transitions, failure modes - src/room/adapters/fake-hue-bridge.ts, fake-light.ts, fake-sensor.ts - tests/room/conformance/: adapter conformance test suite (any adapter must pass)

Disabled: - Real Hue bridge discovery - Real sensor I/O - Any network calls beyond localhost - Any UI rendering work

Dependencies: Phase 6 (environmental scaffolding) frozen contracts

Acceptance: - pnpm bootstrap succeeds on clean macOS install - pnpm doctor reports green on a fresh Mac - pnpm test:room passes full conformance against fake adapter - Fake room emits deterministic events visible in event store contract (next phase will wire storage) - No real network calls anywhere in suite

Closeout verdict target: safe — substrate only, no operational authority

Phase 11 — Local Persistence & Event Store

Purpose: Stand up the append-only event store that every later phase reads/writes through. Read-only projection layer. No UI yet.

Implemented: - SQLite (WAL mode) via better-sqlite3 — chosen for zero-process embedded storage, deterministic for tests - db/migrations/: numbered migrations runnable via pnpm db:migrate - Tables: events (append-only audit), room_events, telemetry_events, replay_traces, runtime_executions, approval_lifecycle, routine_suggestions, model_calls - src/store/event-store.ts: append-only writer, no UPDATE, no DELETE except via retention job - src/store/projections/: read-only projections for room state, recent traces, telemetry rollups, suggestion inbox - src/store/retention.ts: dated retention policies per table (audit = forever, telemetry = 30 days, replay metadata = 90 days) - Schema version recorded with every event; schema migration tests prove backward read compatibility

Disabled: - Any cross-machine sync - Any cloud backup - Any mutating endpoints in projections - Postgres (deferred — SQLite is sufficient for one-room scale)

Dependencies: Phase 10 substrate

Acceptance: - Append-only invariant proven by property test (no code path can UPDATE/DELETE events) - Fake room events from Phase 10 land in store and re-emerge via projections - Replay trace round-trip: write trace → query via projection → identical metadata - Schema migration test: v1 → v2 → v3 → projections still work on v1 data

Closeout verdict target: safe — append-only, no mutation surface, local-only

Phase 12 — Command Center UI Realization (Synthetic + Live Read-Only)

Purpose: Render the Rest/Working/Audit screens against fake-room data and real local metadata projections. No mutating endpoints exposed.

Implemented: - Next.js app (existing scaffold) renders three screens: /rest, /working, /audit - Tauri desktop shell wrapping the Next app for local-only operation (no public network exposure) - Orb rendering: Three.js scene at /rest with idle/listening/working/audit visual states - Cockpit panels at /working: Room Panel (fake devices visible), Project Panel, Cost Panel, Suggestion Inbox, Recent Activity - Audit screen: Trace Timeline (replay metadata, scrubbable), Replay Viewer (metadata-only), Governance Boundary Viewer (static + observed overlay), Runtime Dependency Viewer - Demo Mode: separate synthetic dataset, badge visible in UI, structurally cannot read live tables (separate connection) - Developer Console: hidden by default, env-gated, shows raw projections in JSON - All data flows through the read-only Observability API from Phase 9. No new mutating endpoints introduced.

Disabled: - Run/retry/approve/mutate buttons (forbidden DOM check enforced by test) - Raw frame/voice/prompt rendering (redaction smoke test on every panel) - Public network exposure (Tauri binds to 127.0.0.1 only) - Remote dashboard / web hosting

Dependencies: Phase 9 contracts, Phase 11 store

Acceptance: - All three screens render on a fresh Mac with bootstrap-only data (fake room running) - Demo Mode toggles between synthetic and live, never mixes - UI source/DOM snapshot tests prove no forbidden buttons exist - Redaction tests: every panel passes with forbidden fields injected — they never render - Tauri binary runs entirely offline

Closeout verdict target: safe — read-only observability rendered, no authority surface

Phase 13 — Local Model Runtime & Router Realization

Purpose: Wire real local model inference behind the existing model registry. Cloud providers remain disabled by default.

Implemented: - Ollama as the local runtime (chosen for stability, model breadth, low setup friction, CPU/GPU/Metal support) - src/models/providers/ollama.ts: implements the existing provider interface - src/models/registry.ts: reads config/models/registry.yaml, exposes capability lookup - config/models/registry.yaml example: llama3.2:3b (T1), qwen2.5:7b (T2), claude-haiku (T3, disabled), claude-opus (T4, disabled) - Router integration: existing intent → safety → capability → cost stages now resolve to real providers - Cost telemetry: every model call writes model_calls event with input_tokens, output_tokens, latency_ms, success - Model health check on startup: ollama list, ping each enabled model, fail closed if missing - Fallback policy: T1 fails → T2 attempted with explicit log; T2 fails → user-visible “local models unavailable” message, never silent cloud upgrade

Disabled: - Cloud providers default enabled: `false` in registry - Auto-installation of models (manual `ollama pull` only, documented) - Model finetuning, training, weight modification - vLLM (deferred — Ollama is sufficient for one-user scale)

Dependencies: Phase 11 store (for `model_calls` events)

Acceptance: - Fresh bootstrap + `ollama pull llama3.2:3b` + restart → router routes a real query to local model - Cost telemetry visible in Cockpit Cost Panel - Disabling all models in registry → fail-closed error, no fallback to cloud - Enabling cloud requires explicit `env` flag + registry edit + budget cap configured

Closeout verdict target: `safe` — `local-first`, `cloud opt-in` only

Phase 14 — Voice Runtime Realization (PTT)

Purpose: Implement push-to-talk voice. No wake word. No always-listening. Voice is transport over the text pipeline, not a new authority surface.

Implemented: - Local STT: `whisper.cpp` (or `faster-whisper` if Python preferred — verify latest version before install) for offline transcription - Local TTS: Piper TTS (high-quality, fast, fully offline; verify latest) - Push-to-talk binding: configurable global hotkey via Tauri - Audio capture: only while hotkey held; mic-active indicator (red dot) visible in UI for entire capture duration - Streaming pipeline: audio buffer → Whisper → text → existing intent/safety/router → TTS → playback queue - Barge-in cancellation: user starts speaking again → AbortSignal flushes playback queue, cancels in-flight TTS - Cloud STT/TTS providers (OpenAI Whisper API, OpenAI TTS): registered but disabled, opt-in via budget cap + consent dialog - Voice telemetry: only metadata (`duration_ms`, char count, latency breakdown). No transcripts, no audio in telemetry.

Disabled: - Wake word (“hey jarvis” never enabled in this plan) - Always-listening / background mic - Voice-only approval (approvals still require on-screen click) - Default speaking of tool output, file content, code blocks, audit logs, personal context, error stack traces

Dependencies: Phase 13 model runtime, Phase 11 store

Acceptance: - PTT roundtrip: hold key → speak → text appears → response streamed back via TTS - Releasing key mid-utterance → capture ends cleanly - Barge-in: speaking during TTS playback flushes queue within 200ms - Mic-active indicator visible whenever capture is active (UI source test) - Telemetry events contain zero transcript content (redaction test)

Closeout verdict target: `safe` — voice as transport, no authority elevation

Phase 15 — Vision Runtime Realization (Screenshot + Mock Camera)

Purpose: Wire real vision providers behind Phase 7 contracts. Start with screenshot OCR. Camera remains mock-only until later phase.

Implemented: - Tesseract OCR for printed text (verify latest, fast, offline) - PaddleOCR for harder text (optional, larger model, install only if needed) - YOLOv8n for object detection on screenshots (10MB, fast, runs on CPU) - Screenshot capture: user-initiated only via hotkey or /working panel button. Region selection supported. - Mock camera adapter: emits synthetic frames against the same contract a real camera would - Vision session lifecycle from Phase 7: every capture creates a session, metadata-only replay, raw frames never stored beyond in-memory processing - Cloud vision (Claude/GPT vision): registered, disabled by default, opt-in with budget cap + per-call consent

Disabled: - Real camera (deferred to a later sub-phase after Mac + room arrive, gated behind explicit user action) - Continuous capture / background vision (no frame sampling without user trigger) - Autonomous visual actions (vision is advisory, never an authority surface) - Raw frame storage in telemetry/replay (hash-only identity as per Phase 7)

Dependencies: Phase 7 contracts, Phase 13 model runtime

Acceptance: - Screenshot → OCR → text result rendered in working panel - Mock camera emits frames against contract; tests pass against same contract a real camera will use - No raw frames anywhere in telemetry (redaction test with injected frame bytes — they never appear) - Cloud vision disabled by default; enabling requires `env` flag + budget + per-call consent

Closeout verdict target: `safe` — advisory perception, mock camera, no autonomous capture

Phase 16 — Room Adapter Realization (Fake → Real)

Purpose: Path from fake room to real room. Hue lights are the first real device. One device at a time. Read-only first. Dry-run second. Approval-gated execution third.

Implemented in 16A — Fake Adapter Hardening: - Fake Hue bridge passes every conformance test: state read, state mutate (in-memory), failure modes, stale state, offline simulation, command rejection - Fake adapter becomes the conformance contract. Every real adapter must pass these tests.

Implemented in 16B — Real Hue Read-Only: - `src/room/adapters/hue-adapter.ts`: implements adapter interface, reads bridge state via Hue Bridge v2 API - Manual config: user enters bridge IP and API key in `config/room/hue.yaml`. No auto-discovery in this plan. - Bridge state appears in Room Panel alongside fake devices - All writes return `not_implemented` — read-only mode

Implemented in 16C — Real Hue Dry-Run: - Adapter accepts mutate commands but emits `dry_run_plan` event instead of calling bridge - Plan visible in Cockpit, includes intended state, current state, diff - User confirms plan to enter approval lifecycle

Implemented in 16D — Real Hue Approval-Gated: - Approved plan → adapter calls bridge API → verification read after command → state diff logged - Every real command writes `room_action` audit event with full provenance (who/what/when/which adapter/which approval) - Failure modes: timeout, auth error, bridge offline, partial success all handled with explicit user-visible state

Disabled: - Scenes / macros / multi-device routines (deferred) - Schedules / time-based device actions (deferred) - Cloud Hue Remote API (local bridge only) - Auto-discovery of devices (manual config only initially) - Trust class elevation by voice or by JARVIS itself (only user via config can change trust class)

Dependencies: Phase 10 substrate, Phase 11 store, Phase 12 UI, Phase 16A conformance pass

Acceptance: - 16A: fake adapter passes 100% conformance suite - 16B: real Hue bridge state visible in cockpit, no writes possible - 16C: dry-run plans render correctly, no API write calls executed - 16D: approved command executes, verification read confirms state, audit event captures full trace - Fake-to-real regression test: same conformance suite passes against both

Closeout verdict target: safe – one real device class, three-stage gate, no autonomous control

Phase 17 — Scheduled Assistance Runtime

Purpose: Turn Phase 8 suggestion-only scaffolds into a real scheduler that generates real reports without ever triggering side effects.

Implemented: - Scheduler runs as a foreground process inside the local API (no background daemon) - Scheduled jobs: daily self-audit (8pm local), weekly cost report (Sunday 6pm), project progress summary (Friday 5pm), calibration baseline check (monthly) - Each job reads from event store, writes a routine suggestion event with the report content - Suggestions appear in Cockpit Suggestion Inbox - User actions on suggestions enter the approval lifecycle (Phase 18) — no auto-execution - Kill switch: env flag disables all scheduler runs immediately

Disabled: - Background / headless scheduler (must be foreground until Phase 20) - Auto-execution of suggestions - Routine chaining (one suggestion cannot trigger another) - Self-modifying routines (scheduler cannot edit its own jobs) - Auto-tuning of calibration thresholds

Dependencies: Phase 11 store, Phase 12 UI for inbox rendering

Acceptance: - Daily self-audit produces a report visible in inbox within 60s of scheduled time - Killing scheduler env flag → no new suggestions appear - Suggestion → user click → approval lifecycle entered (no shortcut) - Audit event written for every scheduler tick (whether or not it produced a suggestion)

Closeout verdict target: safe – suggestion-only, foreground, killable

Phase 18 — Approval-Gated Execution Layer

Purpose: The first phase where JARVIS can do something. Tightly bounded. Dry-run-first. Approval lifecycle is the only path.

Implemented: - Approval service: receives execution proposals, presents UI for review, records decision, dispatches to tool runtime - Tool runtime extension: read-only commands from Phase 3.5 become read+write under approval - First write commands: `note.create` (Obsidian), `project.task.create`, `room.action.execute` (gated by Phase 16D) - Each command has: dry-run mode (default), approval requirement (always), rollback hint where applicable - Approval lifecycle: `proposed` → `reviewed` → `approved` | `denied` | `expired` → `executed` → `verified` | `failed` - Approval expiry: 5 minutes default. Re-proposal required after expiry. - Compensation: every executed write has a recorded inverse (where compensatable). Audit shows both the action and the available inverse. - Auto-approval: never. No allow-list of auto-approved actions in this plan.

Disabled: - Auto-approval of any class of action - Voice-only approval (UI click still required) - Multi-step execution graphs (one approved command at a time) - Approval inheritance (approving one command never approves a related one) - Cross-session approval persistence (every session re-requires approval)

Dependencies: Phases 11, 12, 16 (room actions need adapter-gated execution)

Acceptance: - Approval proposal → UI shows full plan with diff → user clicks approve → command executes → verification step runs → audit event records full lifecycle - Denial / expiry / failure paths each tested - No command bypasses approval (property test against the tool runtime registry) - Compensation actions visible in audit, available for user-triggered rollback

Closeout verdict target: safe – approval lifecycle is the only path to side effects

Phase 19 — Fortress Upgrades

Purpose: Add the four high-signal capabilities that turn JARVIS from “impressive system” into “demonstrably governed system you’d trust.” Pure additive layer on top of Phase 10-18.

Implemented:

19A — Architecture Graph + Runtime Dependency Visualizer - Interactive D3/React-Flow rendering of the full system: every phase, every module, every adapter, every governance boundary - Click any node → see what depends on it, what it depends on, current health, recent activity - Two layers: Static (designed dependencies from code) + Observed (actual call paths from telemetry) - Discrepancies highlighted (designed-but-unused, used-but-undesigned) - Replaces the “ARCHITECTURE.md” wall of text as the primary entry for non-developers - Read-only: no node is clickable to execute anything

19B — Live Telemetry / Observability Cockpit - Real-time dashboard: cost per minute, latency per tier, model selection patterns, recent errors, replay trace volume - Drill-down: click a spike → see the specific events - Per-day, per-week, per-month rollups - Anomaly markers: P95 drift, cost spike, error rate elevation - Reads from Phase 11 store via existing Observability API. No new authority.

19C — Governance / Safety Boundary Visualizer - Visual rendering of every safety classification, every approval gate, every adapter trust class - Color-coded by trust class (`observe_only` / `safe_mutate` / `restricted_mutate` / `forbidden`) - “Forbidden edge” tripwire from Phase 9G: if a forbidden call path is ever observed, the relevant edge turns red and stays red until acknowledged - One-click “show me what’s currently disabled” view (renders the disabled-feature matrix visually)

19D — Red-Team / Attack Simulation Layer (CAI Integration) - CAI (Alias Robotics framework) integrated as a JARVIS plugin via the Phase 5 plugin architecture - Runs in sandboxed mode: explicit target whitelist, hard block on anything not whitelisted, full audit trail of every action - Use cases: autonomous security audit of JARVIS itself, CTF practice, controlled vulnerability assessment of user-authorized targets only - Approval-gated: every CAI action class requires per-class authorization (e.g., “approve all read-only reconnaissance against localhost for next 30 minutes”) - Built on the same approval lifecycle (Phase 18) — no parallel authority path - The defensive demo: “JARVIS can attack itself to prove its defenses hold”

Disabled in Phase 19: - CAI against any non-whitelisted target - CAI auto-elevation of trust class - Dashboard mutations (everything read-only) - Public/remote hosting of any of these dashboards

Dependencies: All prior phases. Especially Phase 18 (CAI uses approval lifecycle).

Acceptance: - Architecture graph renders full system, click-throughs work, no execute buttons exist - Telemetry cockpit shows live data within 5 seconds of an event - Forbidden edge tripwire fires when a deliberate test violation is injected - CAI integration completes one full sandboxed audit against localhost-only target with full audit trail - All four dashboards pass DOM forbidden-button test

Closeout verdict target: safe – observability and red-teaming as additive surfaces, no new authority

Phase 20 — Final Integration, Hardening, Packaging

Purpose: Cross-phase closeout. Packaging. Move-in completion. Hardening pass on every authority surface.

Implemented: - Tauri packaging: .dmg for macOS, .AppImage for Linux - Service startup orchestration: single pnpm start boots app, model runtime, scheduler, voice workers - Reset scripts: pnpm reset:db, pnpm reset:room, pnpm reset:all - Install/onboarding documentation: bootstrap, fake-room walkthrough, demo mode, real-Hue onboarding, sensor onboarding, troubleshooting - Cross-phase audit: every authority surface re-tested under penetration-style scenarios (e.g., “if I edited the registry to enable cloud, would any guard fail closed?”) - Performance pass: latency budgets verified, P95 met on M4 hardware - Failure mode tests: bridge offline, model unavailable, scheduler stuck, store full - Final disabled-feature matrix snapshot (docs/disabled.md) — comprehensive list of what is intentionally not enabled

Disabled (and will remain disabled past Phase 20): - Wake word - Always-listening - Background camera - Autonomous device execution - Public/remote dashboards - Voice-only approval - Auto-approval - Whole-home / multi-room (deferred to a hypothetical Phase 21+)

Dependencies: All prior phases

Acceptance: - pnpm package:mac produces a signed .dmg that installs and runs end-to-end - Onboarding doc walks a fresh user from clone to first approved room action in under 30 minutes - Disabled-feature matrix verified by snapshot test (any new feature accidentally enabled fails the test) - All performance budgets met

Closeout verdict target: safe – system operational, hardened, demoable, move-in ready

4. Recommended Software Stack

Layer	Choice	Why	Alternatives Considered	Install When	Local-Only?
Frontend framework	Next.js (existing)	Already in repo, App Router fits the screen model, RSC for read-only projections	Remix, SvelteKit	Already installed	Yes
Desktop shell	Tauri	Rust-based, smaller bundle than Electron, better security defaults, native macOS feel	Electron (heavier, security history)	Phase 12	Yes
Graph visualization	React Flow	Best-in-class for dependency/architecture graphs, programmable, accessible	D3 (lower-level, more work), Cytoscape (heavier)	Phase 12	Yes
Orb / 3D rendering	Three.js + react-three-fiber	Proven, performant, large community	PixiJS (2D), bare WebGL (too low-level)	Phase 12	Yes
Local database	SQLite via better-sqlite3	Zero-process, deterministic, WAL mode for concurrency, perfect for one-user scale	Postgres (overkill, requires daemon), libsql (newer, less battle-tested)	Phase 11	Yes
Event bus	In-process EventEmitter + DB events table	Single-process design needs no broker. DB is the durable bus.	Redis pubsub, NATS (both add a daemon)	Phase 11	Yes
Local model runtime	Ollama	Stable, broad model support, native macOS Metal acceleration, simple install	LM Studio (GUI-first, less scriptable), llama.cpp (lower-level), vLLM (Linux/GPU-centric)	Phase 13	Yes
Cloud model providers	Anthropic SDK, OpenAI SDK	Existing provider abstraction supports both	—	Phase 13 (registered, disabled)	Opt-in
Local STT	whisper.cpp (or faster-whisper)	Fast, offline, accurate; verify latest version before install	OpenAI Whisper API (cloud, opt-in only)	Phase 14	Yes
Local TTS	Piper TTS	High-quality offline voices, fast, small models; verify latest	Coqui TTS (heavier), system TTS (lower quality)	Phase 14	Yes
Cloud STT/TTS	OpenAI realtime / TTS APIs	For higher-quality voice once budget permits	—	Phase 14 (registered, disabled)	Opt-in
OCR (printed)	Tesseract via node-tesseract-ocr	Mature, fast, offline	PaddleOCR (better but heavier)	Phase 15	Yes
OCR (harder cases)	PaddleOCR	Better on small/skewed/handwritten text	TrOCR (transformer, slower)	Phase 15 (optional)	Yes
Object detection	YOLOv8n via ultralytics Python sidecar	Small (10MB), fast, accurate enough for desk use	MediaPipe (different use case), DETR (slower)	Phase 15	Yes
Pose/gesture	MediaPipe	Best-in-class for hands/face/body, JS bindings	OpenPose (heavier)	Deferred (post-Phase 20)	Yes
Vector database	LanceDB or sqlite-vec	Embedded, no daemon, fits one-user scale	Qdrant (Docker), Chroma (heavier), Pinecone (cloud)	Phase 13 if memory bridge needs it	Yes
Embeddings	Nomic Embed via Ollama, or all-MiniLM via transformers.js	Local, free, good enough quality	OpenAI text-embedding-3-small (cloud, opt-in)	Phase 13	Yes
Process supervisor	pnpm scripts + Tauri	Single-process design	PM2, systemd (both	Phase 12	Yes

Logging	pino + rotating file transport	needs no PM2 Fast, structured, simple	overkill) winston (heavier), bunyan (older)	Phase 11	Yes
Test tooling	Vitest (existing), Playwright for UI E2E	Already in repo for Vitest; Playwright is standard for desktop UI E2E	Jest (slower), Cypress (heavier)	Phase 12 adds Playwright	Yes
Packaging	Tauri bundler	Native to chosen shell	Electron-builder Diyhue (simulates Hue but adds Python daemon), shelly emulator	Phase 20	Yes
Fake device simulation	Custom in-process JS adapters	Zero dependencies, deterministic, fast tests		Phase 10	Yes
Real Hue integration	node-hue-api or direct Hue Bridge v2 REST	Mature, well-documented, local bridge only	Hue Sync Box API (cloud)	Phase 16B	Yes
Sensor integration	Node Bluetooth / USB libraries per sensor class	Driven by which sensors get purchased	Home Assistant bridge (heavyweight)	Deferred until sensors arrive	Yes
Red-team layer	CAI (Alias Robotics) via plugin architecture	Best-in-class autonomous security testing framework	Manual pentesting scripts	Phase 19D	Yes (sandboxed)

5. Installation Roadmap

Group 1 — Install Immediately (Current Machine, Before MacBook)

Package	Purpose	Verification
Node 22 LTS, pnpm	Existing dev environment	node -v, pnpm -v
better-sqlite3 (npm)	Prepares Phase 11 substrate	pnpm test:store after Phase 11 starts
React Flow, Three.js, react-three-fiber (npm)	Prepares Phase 12 UI	Unit tests render skeleton
Vitest (existing)	Test runner	pnpm test passes

Smoke test: pnpm bootstrap && pnpm test && pnpm doctor — all green.

Group 2 — Install When Preparing the MacBook

Package	Purpose	Verification
Homebrew	macOS package manager	brew --version
Tauri CLI + Rust toolchain	Desktop shell	cargo --version, pnpm tauri info
Ollama	Local model runtime	ollama --version, ollama list
Playwright + browsers	E2E UI tests	pnpm test:e2e

Smoke test: Clone repo → pnpm bootstrap → pnpm dev → /rest orb renders.

Group 3 — Install After UI Rendering Starts (Phase 12)

Package	Purpose	Verification
Tauri build dependencies	Desktop bundling	pnpm tauri build succeeds
Pino + rotating-file-stream	Logging	Log file created with structured events

Group 4 — Install After Persistence/Event Store Exists (Phase 11)

Package	Purpose	Verification
sqlite-vec or LanceDB	Vector storage for memory bridge	Embedding write/read round-trip
Nomic Embed via Ollama	Local embeddings	ollama pull nomic-embed-text

Group 5 — Install When Model Runtime Begins (Phase 13)

Package	Purpose	Verification
ollama pull llama3.2:3b	T1 local model	ollama run llama3.2:3b "hello"
ollama pull qwen2.5:7b	T2 local model (verify latest)	ollama run qwen2.5:7b "hello"
Anthropic SDK (npm, already present)	Cloud provider (disabled)	Connection test only

Group 6 — Install When Voice Realization Begins (Phase 14)

Package	Purpose	Verification
whisper.cpp or faster-whisper (verify latest)	Local STT	Transcribe test audio file
Piper TTS (verify latest)	Local TTS	Synthesize test phrase
Tauri global hotkey plugin	PTT binding	Hotkey triggers capture event

Group 7 — Install When Vision Realization Begins (Phase 15)

Package	Purpose	Verification
node-tesseract-ocr + Tesseract binary	Screenshot OCR	OCR a known image, expected text returned
ultralytics (Python sidecar)	YOLOv8n	Detect objects in test image
YOLOv8n weights	Object detection model	First inference completes

Group 8 — Install When Fake Room Simulation Begins (Phase 10)

Package	Purpose	Verification
(No external packages — fake adapters are pure JS)		pnpm test:room:conformance passes

Group 9 — Install When Real Hue/Device Integration Begins (Phase 16B)

Package	Purpose	Verification
node-hue-api Bridge IP + API key	Hue Bridge v2 client Manual config	Bridge state read returns valid JSON Config file validated by schema test

Group 10 — Install Only After Approval-Gated Execution Exists (Phase 18)

Package	Purpose	Verification
Compensation tracking utility (custom)	Rollback inverse generation	Inverse of test command computed and recorded

Group 11 — Install for Fortress Phase (Phase 19)

Package	Purpose	Verification
CAI framework (Python sidecar) Recharts or visx	Red-team layer Telemetry cockpit charts	Sandboxed run against localhost succeeds Cost chart renders against synthetic data

Defer Indefinitely

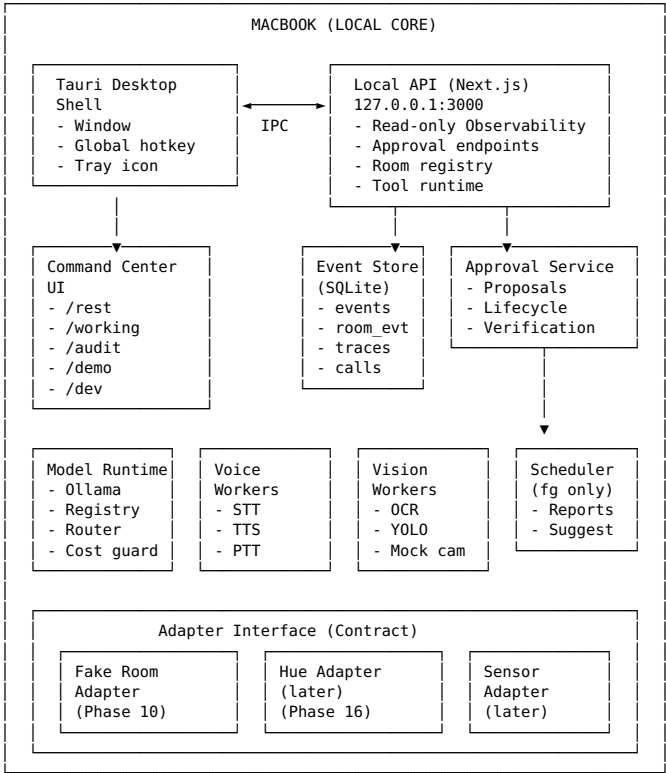
- vLLM (only useful at scale)
- Postgres (SQLite covers one-room)
- Redis (in-process event bus is sufficient)
- PM2 / systemd (Tauri handles lifecycle)
- Home Assistant (bypasses governance)
- Hue Cloud Remote API (local-only invariant)

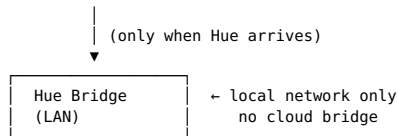
MacBook Arrival Checklist

1. Install Homebrew
2. brew install node@22 pnpm rust ollama
3. git clone <repo>
4. pnpm bootstrap
5. pnpm doctor → green
6. pnpm test → all 1772+ pass
7. pnpm dev → Tauri opens, /rest orb renders
8. pnpm demo → demo mode active, synthetic room visible
9. ollama pull llama3.2:3b
10. Restart, verify model call lands in cost panel
11. **No real hardware required to complete this checklist.**

When Hue lights arrive: add bridge config, run read-only test, single light dry-run, approved command, audit verification.

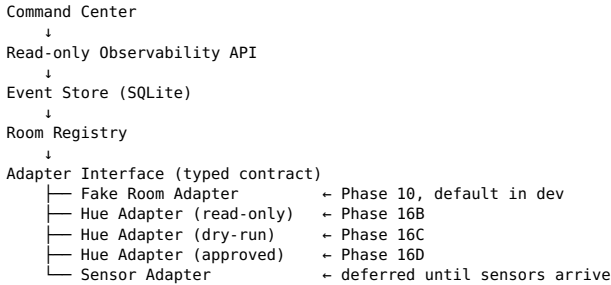
6. Target Runtime Topology





Process boundaries: - Single Tauri app process hosts the Next.js server, the Ollama client, the SQLite handle, the voice/vision workers (as web workers), and the scheduler (foreground only) - Ollama runs as a separate user-installed daemon, accessed via localhost:11434 - Vision Python sidecar (for YOLO) optional, only when Phase 15 ships, communicates via stdio - No process exposed beyond 127.0.0.1 ever

Fake-to-real adapter topology:



The contract is the same at every stage. The fake adapter is the conformance suite that every real adapter must pass before it can be enabled.

7. Data Flow Architecture

User Text Input

- **Allowed:** typed text from local UI
- **Forbidden:** unsolicited input from network, file watchers, external APIs
- **Stored:** message ID, hashed user ID (single user system, but consistent), timestamp, model_id, latency, token counts
- **Never stored:** raw prompt content beyond session, response content beyond session window
- **Audit events:** message_received, intent_classified, safety_classified, model_called, response_streamed
- **Approval gates:** none (text input is not a side effect)
- **Failure:** user sees error, no silent retry on cloud

Push-to-Talk Voice Input

- **Allowed:** audio captured only while hotkey held, mic-active indicator visible
- **Forbidden:** background capture, wake-word detection, always-on listening
- **Stored:** duration_ms, character count of transcript, latency breakdown
- **Never stored:** raw audio, raw transcript
- **Audit events:** voice_capture_started, voice_capture_ended, stt_completed, then text flow takes over
- **Approval gates:** none for transport; voice cannot bypass on-screen approval for actions
- **Failure:** capture ends, user sees “voice unavailable”, no silent fallback to cloud STT

Model Routing

- **Allowed:** routed input from text or voice transport
- **Forbidden:** routing without prior safety classification, routing to cloud without budget approval
- **Stored:** routing decision, tier, model_id, reason, cost estimate, latency
- **Never stored:** prompt body in telemetry (only token counts)
- **Audit events:** route_decided, model_called, model_response_streamed
- **Approval gates:** cloud escalation above budget cap requires consent
- **Failure:** route fails closed, user-visible error

Tool Proposal

- **Allowed:** model outputs tool_use intent
- **Forbidden:** auto-execution
- **Stored:** proposal_id, tool_name, dry-run plan, current state
- **Never stored:** proposal body if it contains secrets (redacted via shared redaction lib)
- **Audit events:** tool_proposed, dry_run_computed
- **Approval gates:** every tool proposal requires user click
- **Failure:** proposal expires after 5 minutes, must be re-proposed

Approval-Gated Execution

- **Allowed:** explicitly user-approved tool calls only
- **Forbidden:** auto-approval, voice-only approval, inheritance
- **Stored:** full approval lifecycle, tool_call, result, verification, audit_event_id
- **Never stored:** secrets, raw payloads of execution body
- **Audit events:** approval_proposed, approval_granted, tool_executed, verification_completed
- **Approval gates:** the entire flow is the gate
- **Failure:** rollback compensation triggered where defined, user-visible

Fake Room Device Action

- Same flow as real action, just with fake adapter

- All audit events written
- Allows the entire pipeline to be tested without hardware

Real Hue Light Action (when enabled)

- **Allowed:** approved command via Phase 16D adapter
- **Forbidden:** unapproved commands, bypass of dry-run stage
- **Stored:** intended state, current state, diff, command sent, verification state read, audit_event_id
- **Never stored:** Hue API key in audit (referenced by config ID only)
- **Audit events:** room_action_proposed, dry_run_plan, approved, command_sent, verification_read, result
- **Approval gates:** every command, every time
- **Failure:** bridge unreachable → fail-closed, user sees error, no retry

Sensor Observation (when sensors exist)

- **Allowed:** read-only sensor state into store
- **Forbidden:** sensor data triggering device actions without approval
- **Stored:** sensor_id, observation, timestamp, freshness_ts
- **Never stored:** raw audio/video from sensor; only derived state
- **Audit events:** sensor_observed, sensor_stale, sensor_offline
- **Approval gates:** sensor-triggered actions require explicit approval workflow (no auto-routines)
- **Failure:** sensor offline → state goes to “unknown”, presence model defaults to unknown-safe

Vision Observation Flow

- **Allowed:** screenshot OCR triggered by user, mock camera frame in dev
- **Forbidden:** real camera capture in this plan, background frame sampling
- **Stored:** frame metadata, OCR text result (if user-initiated), session_id, hash-only frame identity
- **Never stored:** raw frame bytes
- **Audit events:** vision_session_started, frame_processed, ocr_completed
- **Approval gates:** vision can never trigger device actions on its own
- **Failure:** OCR unavailable → user sees error, no silent cloud upgrade

Project Indexing Flow

- **Allowed:** user-triggered pull-only index of declared project paths
- **Forbidden:** background file watching, silent indexing, cross-project synthesis
- **Stored:** project_id, file hashes, derived state metadata
- **Never stored:** raw file bodies in telemetry; vector embeddings in dedicated table
- **Audit events:** project_index_started, project_indexed, derived_state_updated
- **Approval gates:** writes to projects (e.g., new task) go through approval lifecycle
- **Failure:** index fails → derived state marked stale, user sees indicator

Scheduled Self-Audit Flow

- **Allowed:** scheduled read-only queries against event store
- **Forbidden:** schedule-triggered side effects, schedule-triggered cloud calls
- **Stored:** suggestion content, scheduled_run_id, projection snapshots
- **Never stored:** raw event bodies (rolled-up metadata only)
- **Audit events:** routine_tick, routine_executed_read_only, suggestion_emitted
- **Approval gates:** any suggestion → user action → enters approval lifecycle
- **Failure:** routine fails → logged, no retry, no escalation

Command Center Projection

- **Allowed:** read-only queries via Observability API
- **Forbidden:** any mutating call from UI
- **Stored:** nothing; UI is presentation-only
- **Audit events:** none (read-only)
- **Failure:** projection unavailable → panel shows error state, no fallback to live mutating call

Demo Mode Projection

- **Allowed:** reads only from synthetic_dataset
- **Forbidden:** any read from live tables, any write anywhere
- **Stored:** nothing
- **Audit events:** demo_mode_entered, demo_mode_exited
- **Failure:** synthetic data missing → demo mode refuses to enter

8. Room OS Model

Core Entities

```
type RoomProfile = {
  id: string;
  name: string;
  zones: Zone[];
  devices: Device[];
  sensors: Sensor[];
  policies: RoomPolicy[];
  retention: RetentionPolicy;
};

type Zone = {
  id: string;
```

```

    name: string;
    kind: 'desk' | 'bed' | 'door' | 'ambient' | 'custom';
};

type Device = {
    id: string;
    zone_id: string;
    kind: 'light' | 'plug' | 'led_strip' | 'speaker' | 'display';
    capabilities: Capability[];
    adapter: AdapterRef;
    trust_class: TrustClass;
    state: DeviceState;
};

type Capability = 'on_off' | 'dim' | 'color' | 'temperature' | 'meter';

type AdapterRef = {
    kind: 'fake' | 'hue' | 'tapo' | 'nanoleaf' | ...;
    config_ref: string;
};

type TrustClass = 'observe_only' | 'safe_mutate' | 'restricted_mutate' | 'forbidden';

type DeviceState = {
    value: unknown;
    observed_at: timestamp;
    freshness: 'fresh' | 'stale' | 'offline' | 'unknown';
};

type Sensor = {
    id: string;
    zone_id: string;
    kind: 'motion' | 'presence' | 'temperature' | 'humidity' | 'light_level';
    trust_class: TrustClass;
    state: SensorState;
};

type RoomPolicy = {
    id: string;
    description: string;
    applies_to: string[];
    rules: PolicyRule[];
};

type ApprovalPolicy = {
    device_kind: string;
    capability: Capability;
    requires_approval: boolean;
    expiry_ms: number;
};

type RetentionPolicy = {
    audit_events: 'forever';
    telemetry: '30d';
    replay_metadata: '90d';
    derived_state: 'forever_with_versioning';
};

type DemoRoomProfile = RoomProfile & { is_demo: true };

```

Default trust class for any new device: observe_only. Elevation requires explicit config edit by the user; JARVIS cannot self-elevate.

Example First Room (default)

```

# config/room/default-room.yaml
id: jarvis-room-1
name: "Bedroom Workspace"
zones:
  - { id: desk, name: "Desk", kind: desk }
  - { id: bed, name: "Bed", kind: bed }
  - { id: door, name: "Door", kind: door }
  - { id: ambient, name: "Ambient", kind: ambient }
devices:
  - { id: desk_lamp, zone_id: desk, kind: light, capabilities: [on_off, dim], adapter: fake, trust_class: safe_mutate }
  - { id: bed_lamp, zone_id: bed, kind: light, capabilities: [on_off, dim], adapter: fake, trust_class: safe_mutate }
  - { id: led_strip, zone_id: ambient, kind: led_strip, capabilities: [on_off, dim, color], adapter: fake, trust_class: safe_mutate }
  - { id: smart_plug, zone_id: desk, kind: plug, capabilities: [on_off, meter], adapter: fake, trust_class: observe_only }
sensors:
  - { id: motion_door, zone_id: door, kind: motion, trust_class: observe_only }
  - { id: presence_desk, zone_id: desk, kind: presence, trust_class: observe_only }
policies:
  - description: "Night mode warm dim"
    applies_to: [ambient]
    rules: [{ if: time_after_22, then: temperature_warm_and_dim_below_30 }]
retention: default

```

Every device begins as adapter: fake. Moving to real Hue is a single line change in this file plus the adapter conformance test must pass.

9. Fake Device / Fake Hue Strategy

The fake adapter is **the contract**. Its existence is not a stopgap — it is the conformance harness that every real adapter must pass before being allowed near the real bridge.

Fake Hue Bridge

- In-memory state: lights, groups, scenes, schedules
- API surface mirrors Hue Bridge v2 endpoints (so the real adapter swap is a one-line change)
- Deterministic transitions: setting brightness 0→100 takes a configurable simulated time
- Persisted between sessions (so dev workflow doesn't reset every restart)

Fake Light Device

- Properties: on, brightness, color, temperature, last_command_at, last_command_result
- Responds to: turn on/off, set brightness, set color, set temperature
- Returns realistic latency (50-200ms simulated)

Fake Failure Modes (essential for conformance)

Failure	Simulation
Online → offline	Toggle via dev console or scheduled in test fixture
Stale state	Last observed_at exceeds freshness threshold
Command timeout	Adapter holds reply for >5s, then fails
Auth error	Adapter returns 401 (simulated invalid API key)
Partial success	Set color succeeds, set brightness fails on same call
Approval denied	Approval lifecycle denies before adapter is reached

Fake Telemetry

- Every fake action emits the same telemetry shape a real action would
- Replay traces work identically against fake and real adapters
- Cockpit shows fake events with a “fake” badge — never silently mixed with real

Conformance Suite

Located at tests/room/conformance/. Any new adapter must pass every test. The test file structure:

```
tests/room/conformance/  
  read-state.test.ts  
  command-on-off.test.ts  
  command-dim.test.ts  
  command-color.test.ts  
  failure-offline.test.ts  
  failure-stale.test.ts  
  failure-timeout.test.ts  
  failure-auth.test.ts  
  approval-flow.test.ts  
  audit-trail.test.ts  
  verification-read.test.ts  
  rollback-compensation.test.ts
```

The fake adapter passes 100% of these. When the real Hue adapter ships, it must also pass 100% before Phase 16D enables real writes.

10. Real Hue / Real Device Integration Strategy

Sequencing

1. **16A — Fake Hardening:** fake passes full conformance
2. **16B — Real Read-Only:** real bridge connects, state visible, writes return not_implemented
3. **16C — Real Dry-Run:** writes accepted but only emit dry-run plans, never sent to bridge
4. **16D — Real Approval-Gated:** approved plans execute, verification read confirms state

Rules for Real Hue Integration

- **No auto-discovery initially.** User enters bridge IP + API key in config/room/hue.yaml.
- **Read-only first.** Bridge state visible in cockpit before any write capability exists.
- **One light first.** When write capability ships, exactly one light is enabled; others remain observe_only until manually elevated.
- **Dry-run before execute.** Every write goes through a dry-run stage that produces a visible plan.
- **Approval-gated.** Every real command, every time, requires UI approval.
- **No multi-device scenes.** Phase 16D enables individual device commands only. Scenes are a later phase.
- **No schedules.** Time-based device actions are deferred (these would require a different approval model).
- **No cloud bridge.** Local Hue Bridge v2 only. Hue Remote API explicitly disabled.
- **No voice trust-class changes.** Only the user, via config edit, can change a device's trust class.
- **No policy edits by JARVIS.** Policies are user-owned, file-backed, version-controlled.
- **Audit entry for every real command.** Full provenance: who, what, when, which adapter, which approval ID, intended state, actual state.
- **Verification read after every command.** Confirms the bridge actually reached the intended state.
- **Safe failure behavior.** Bridge unreachable → user sees error, no retry, no degradation to a different path.

Real Sensor Integration (When Sensors Arrive)

- Same path as Hue: fake → conformance → read-only → no writes from sensors trigger device actions without approval
- Default trust class: observe_only forever (sensors should never be trusted to mutate)
- Presence inference remains conservative: present | absent | unknown, with unknown as the safe default

11. Authority Surface Map

Subsystem	Read	Write	Execute	Approve	Schedule	Network	Cam/Mic	Room Devices	Approval Req	Disabled by Default	Notes
-----------	------	-------	---------	---------	----------	---------	---------	--------------	--------------	---------------------	-------

Command Center	✓	✗	✗	✗	✗	✗	✗	✗	n/a	No	Read-only UI; no buttons that mutate
Developer Console	✓	✗	✗	✗	✗	✗	✗	✗	n/a	Yes (env-gated)	Dev-only view of raw projections
Demo Mode	✓ (synthetic)	✗	✗	✗	✗	✗	✗	✗	n/a	No	Cannot read live tables
Fake Room Adapter	✓	✓ (in-memory)	✓ (simulated)	✗	✗	✗	✗	✓ (fake)	Yes	No	Conformance harness
Hue Adapter (read-only)	✓	✗	✗	✗	✗	✓ (LAN)	✗	✓ (read)	n/a	Yes initially	Phase 16B
Hue Adapter (dry-run)	✓	✗ (no bridge writes)	✗	✗	✗	✓ (LAN)	✗	✓ (plan only)	n/a	Yes initially	Phase 16C
Hue Adapter (approved)	✓	✓ (bridge)	✓ (approved)	✗	✗	✓ (LAN)	✗	✓	Yes (every command)	Yes initially	Phase 16D
Sensor Adapter (future)	✓	✗	✗	✗	✗	✓ (LAN)	✗	✗	n/a	Yes	observe_only forever
Voice (PTT)	✓ (mic when held)	✗	✗	✗	✗	✗ (local STT/TTS)	✓ (gated)	✗	n/a	Partly (PTT enabled, wake word never)	Transport only
Vision (screenshot)	✓ (user-triggered)	✗	✗	✗	✗	✗	Screen yes; cam no	✗	n/a	Cam disabled	Advisory only
Vision (camera)	✗	✗	✗	✗	✗	✗	✗	✗	n/a	Yes (deferred)	Mock only in Phase 15
Scheduler	✓	✓ (suggestions)	✗	✗	✓	✗	✗	✗	n/a	No (foreground only)	Emits suggestions, no side effects
Environment (room registry)	✓	✓ (state from adapters)	✗	✗	✗	✗	✗	✗	n/a	No	Owens derived state, never canonical
Project Intelligence	✓	✓ (derived state only)	✗	✗	✗	✗	✗	✗	n/a	No	No silent writes, no auto-promotion
Model Router	✓	✓ (model_calls events)	✓ (model inference)	✗	✗	✓ (Ollama localhost; cloud opt-in)	✗	✗	n/a	Cloud disabled	Local-first default
Tool Runtime (read)	✓	✗	✓ (read-only commands)	✗	✗	✗	✗	✗	No (allowlist)	No	Phase 3.5 commands
Tool Runtime (write)	✓	✓ (approved only)	✓ (approved only)	✗	✗	✗	✗	✓ (via adapter)	Yes (always)	Yes until Phase 18	New writes require approval
Approval Service	✓	✓ (lifecycle records)	✗	✓ (records user decisions)	✗	✗	✗	✗	n/a	No	Only path to side effects
Memory Bridge	✓	✓ (governed mutations only)	✗	✗	✗	✗	✗	✗	Yes (writes)	Partial	Reads budgeted, writes are tool calls
Local Providers (Ollama, Whisper, Piper, Tesseract, YOLO)	✓ (their inputs)	✗	✓ (their inference)	✗	✗	✗ (localhost)	per-provider	✗	n/a	No	Local-only
Cloud Providers (Anthropic, OpenAI, etc.)	✓ (when called)	✗	✓ (when called)	✗	✗	✓ (cloud APIs)	✗	✗	Budget + consent	Yes	Opt-in only
CAI Plugin (Phase 19D)	✓	✗ (sandboxed)	✓ (whitelist only)	✗	✗	✓ (whitelist only)	✗	✗	Yes (per-class)	Yes	Red-team only, sandboxed
Architecture Graph (19A)	✓	✗	✗	✗	✗	✗	✗	✗	n/a	No	Read-only viewer
Telemetry Cockpit (19B)	✓	✗	✗	✗	✗	✗	✗	✗	n/a	No	Read-only dashboard
Governance Visualizer (19C)	✓	✗	✗	✗	✗	✗	✗	✗	n/a	No	Read-only viewer

The pattern: Almost nothing has write authority. Where write exists, approval is required. Where execute exists, it's tightly scoped

12. Governance Invariants to Preserve

Frozen from Phases 1-9. Must remain frozen through Phase 20 and beyond.

- Voice is not an authority surface
- No wake word
- No always-listening
- No voice-only approval
- Derived state is not canonical truth
- No silent writes
- No background indexing unless explicitly introduced with governance (Phase 20 still disables this)
- No autonomous routines
- No raw frames in telemetry
- No raw OCR text in telemetry
- No raw voice in telemetry
- No raw project bodies in telemetry
- No executable replay graphs
- Observability UI is read-only
- Demo mode cannot touch live data
- Developer console is dev-only (env-gated)
- Approval lifecycle remains the only path to side effects
- Environment actions are approval-gated
- Room devices default `observe_only`
- Fake adapters before real adapters (every device class)
- Real devices introduced one at a time
- Frozen phases are substrate, not redesign targets

Added in this plan (will be enforced by Phase 20 tests):

- No public network exposure (Tauri binds to 127.0.0.1)
 - No remote dashboard / web hosting
 - No auto-discovery of devices in Phase 16
 - Cloud providers disabled by default in registry
 - CAI plugin sandboxed to explicit target whitelist
 - Forbidden DOM elements (run/retry/approve buttons) checked by snapshot test
-

13. Iron-Man JARVIS Capability Trajectory

What can be realistically approximated by Phase 20

- **Fast conversational UX:** PTT voice loop, streaming responses, sub-2s first-token target via local models
- **Visible Command Center:** Rest orb + Working cockpit + Audit timeline
- **Room awareness:** declared devices visible, observed state in cockpit, presence model (conservative)
- **Voice interaction:** push-to-talk, barge-in, mic-active indicator
- **Visual perception:** user-triggered screenshot OCR, object detection on selected regions
- **Project memory/context:** Phase 5 derived state + memory bridge surfaces project continuity
- **Replayable reasoning traces:** Audit screen, replay viewer, governance overlay
- **Safe tool use:** approval-gated execution with dry-run plans
- **Ambient room control with approval:** lights/plugs respond to approved commands, never autonomously

What requires further local model/runtime maturity (beyond Phase 20)

- Sub-500ms voice round-trip on local-only stack (currently realistic at 1-2s)
- Complex multi-step planning at frontier quality on local models (currently T3/T4 cloud-only)
- Long-context reasoning over the full Obsidian vault locally (memory + retrieval works, but quality is calibration-dependent)

What requires hardware (post-Phase 20)

- Real camera awareness (deferred)
- Real presence detection via mmWave (Aqara FP2 class hardware)
- Ambient audio cues (HomePod / local speaker)
- Multi-room expansion

What requires approval-gated execution (Phase 18 enables)

- Sending messages, posting to apps, modifying files, real device commands

What should remain forbidden (in this plan and forever)

- Wake word / always-listening
- Voice-only approval of any side effect
- Autonomous device routines
- Background camera capture
- Public/remote dashboards
- Auto-approval of any action class
- Self-modifying routines or self-elevating trust classes

What should be deferred indefinitely

- Whole-home automation
- Multi-user / multi-tenant operation
- Sovereign / multi-device sync of memory

- Robotics control
- Neural interfaces (these are not within architectural scope)

The Iron-Man feel comes from: speed, visibility, voice, room awareness, project context, and safety — not from autonomy. Movie-JARVIS does what Tony Stark approves. So does this one.

14. Risk Register

Risk	Likelihood	Impact	Mitigation	Test/Audit Coverage	First Appears
Authority creep	Medium	Critical	Closeout gates per phase; DOM forbidden-button tests; property tests on tool runtime	UI snapshot tests, runtime guard tests	Phase 12
Raw data leakage	Medium	Critical	Shared redaction lib at every boundary; injection tests with known-bad data	Redaction tests on every panel + telemetry sink	Phase 11
Background process drift	Low	High	No background processes by design; scheduler is foreground only; kill switch	Process supervision tests, no-daemon assertions	Phase 17
Provider cost runaway	Medium	High	Hard budget caps in registry; per-call estimates; cloud disabled by default	Cost guard tests, budget cap regression tests	Phase 13
Local model instability	Medium	Medium	Health check on startup; fail-closed; calibration tests per task class	Model health smoke tests, calibration suite	Phase 13
GPU/resource exhaustion	Low	Medium	Single-instance model load; configurable quantization; memory budget alarm	Performance tests, latency budget assertions	Phase 13
Voice privacy leak	Low	Critical	No transcript in telemetry; no audio storage; mic indicator always visible	Redaction tests, audio-not-stored property tests	Phase 14
Vision capture leak	Low	Critical	No raw frames stored; hash-only identity; mock camera default	Frame-redaction tests	Phase 15
Fake-to-real adapter mismatch	Medium	High	Same conformance suite for both; regression test on every adapter	Conformance suite, fake-vs-real diff tests	Phase 16
Hue bridge / network quirks	Medium	Medium	Local-only bridge; explicit failure modes; verification reads	Failure-mode tests, offline simulation	Phase 16B
Accidental room device action	Low	High	Dry-run first, approval second, verification third	Approval lifecycle tests	Phase 16D
Remote dashboard exposure	Low	Critical	Tauri binds 127.0.0.1; firewall rule; no public route	Network exposure tests at packaging	Phase 12
Demo mode leaking live data	Low	High	Separate DB connection; synthetic-only data source; structural impossibility, not policy	Demo isolation tests	Phase 12
Replay becoming executable	Low	Critical	Replay graphs typed as data only; no “execute” affordance anywhere	Replay non-executability tests	Phase 12
Scheduler becoming autonomous	Low	Critical	Foreground only; kill switch; <code>side_effects_allowed=false</code> invariant test	Scheduler boundary tests	Phase 17
Environmental action safety	Medium	High	Trust classes; <code>observe_only</code> default; approval per command	Trust-class enforcement tests	Phase 16
Project memory corruption	Low	High	Append-only event store; derived state rebuildable from events; backup before migration	Append-only property tests, projection rebuild tests	Phase 11
MacBook setup drift	Medium	Medium	Bootstrap script with version pins; doctor command verifies environment	CI on macOS, doctor command	Phase 10
OS permission issues (mic/screen)	Medium	Medium	Explicit permission requests at first use; clear UX when permission denied	OS permission integration tests	Phases 14, 15
CAI escape from sandbox	Low	Critical	Whitelist enforced at adapter layer; audit every action; deny by default	CAI sandbox boundary tests	Phase 19D

15. Testing and Audit Strategy

Test Layers

Layer	What	When	Bar
Unit tests	Single functions, single modules	Every commit	100% pass
Integration tests	Module interactions (adapter↔store↔projection)	Every commit	100% pass
Property tests	Append-only invariants, no-mutation invariants, redaction invariants	Every commit	100% pass
Snapshot/schema tests	UI DOM snapshots, telemetry schema, audit event schema	Every commit	100% pass (intentional diffs require explicit approval)
Redaction tests	Inject known-bad payloads into every telemetry/UI boundary	Every commit	Zero leaks
No-mutation proof tests	Read-only endpoints cannot be coerced into mutation	Every commit	100% pass
Disabled-feature guards	Wake word, always-listening, etc. fail closed when invoked	Every commit	100% pass
Replay non-executability tests	Replay graph cannot trigger actions	Every commit	100% pass
UI source/DOM checks	Forbidden buttons (run/retry/approve in read-only views) do not exist	Every commit	Zero forbidden elements
Provider mock tests	All providers behave correctly when mocked	Every commit	100% pass
Fake adapter conformance	Every fake adapter passes the full suite	Every commit	100% pass
Fake-to-real adapter compatibility	Same suite passes against real adapter (when adapter exists)	When real adapter ships	100% pass before enabling
Local provider smoke tests	Ollama/Whisper/Tesseract/YOLO produce expected outputs	Every commit (with mock for absent providers)	100% pass
Room device dry-run tests	Every commit class produces a valid dry-run plan	Every commit	100% pass
Approval-gated action tests	No path bypasses approval	Every commit	100% pass
Performance/latency tests	P50/P95 budgets met on target hardware	Phase closeouts	Within budget
Privacy tests	No raw payloads stored anywhere	Every commit	Zero violations
Closeout gates	Per-phase verification of all invariants	End of every phase	100% pass before freeze

Expected test bar by phase

Phase	Expected total tests (cumulative)	Notes
Phase 9 (current)	1,772	baseline
Phase 10	~2,000	Room substrate + conformance suite
Phase 11	~2,150	Persistence + projections + retention
Phase 12	~2,500	UI rendering + E2E + Tauri + DOM forbidden checks
Phase 13	~2,650	Model runtime + cost guard regressions
Phase 14	~2,800	Voice + privacy + barge-in
Phase 15	~2,950	Vision + redaction + mock camera
Phase 16	~3,200	Real adapter conformance + dry-run + approval
Phase 17	~3,300	Scheduler + kill switch
Phase 18	~3,500	Approval lifecycle + compensation
Phase 19	~3,750	Fortress dashboards + CAI sandbox + graph
Phase 20	~4,000	Final hardening + packaging + disabled matrix snapshot

16. Implementation Style (Codex Prompt Guidance)

Carry forward the same style from Phases 1-9.

- **Small linear slices.** Each Codex prompt produces one PR-sized change.
- **Scaffold-first.** Types and contracts before logic. Logic before integration.
- **Fake-first.** Every adapter ships with a fake.
- **Adapter-first.** Every external dependency hidden behind an adapter interface.
- **Explicit DO NOT implement sections.** Each Codex prompt names what is deferred.
- **Explicit invariants.** Each prompt names the invariants it must preserve.
- **Explicit tests.** Each prompt names the tests that must pass.
- **No broad rewrites.** Surgical edits only. Refactors require their own phase.
- **No re-auditing frozen phases.** Cite them; don't reopen.
- **Mock-first / provider-later.** Real provider integration goes behind disabled flags.
- **Hardware integration behind fake adapter parity.** Real Hue cannot ship until fake passes conformance.
- **Commit after every slice.** Per-slice commits with phase prefix.
- **Closeout audit after every phase.** No phase is "done" until the closeout gate passes.

17. First 15 Codex Slices

Slice 10A.1 — Mac Bootstrap Contract

- **Goal:** Define the bootstrap contract. No execution yet.
- **Files:** `scripts/bootstrap.ts`, `scripts/bootstrap-contract.ts`, `tests/bootstrap.test.ts`
- **Tests:** contract validates required env vars, node version, available memory, disk space
- **Do not:** execute installs, modify the system
- **Commit:** Phase 10A.1: bootstrap contract scaffold

Slice 10A.2 — Local Environment Checker (doctor)

- **Goal:** Implement `pnpm doctor` to report environment status.
- **Files:** `scripts/doctor.ts`, `src/bootstrap/checks/`, `tests/doctor.test.ts`
- **Tests:** reports node version, OS, memory, disk, port availability, missing env vars
- **Do not:** auto-fix anything
- **Commit:** Phase 10A.2: local doctor command

Slice 10B.1 — Room Profile Schema

- **Goal:** Type the `RoomProfile` / `Zone` / `Device` / `Capability` / `TrustClass` schemas.
- **Files:** `src/room/types.ts`, `src/room/schema.ts`, `tests/room/schema.test.ts`
- **Tests:** schema validates known-good profiles; rejects known-bad
- **Do not:** wire any adapter yet
- **Commit:** Phase 10B.1: room profile schema

Slice 10B.2 — Fake Room Registry

- **Goal:** Load `config/room/default-room.yaml` into typed `RoomProfile`.
- **Files:** `src/room/registry.ts`, `config/room/default-room.yaml`, `tests/room/registry.test.ts`
- **Tests:** loads default room, exposes `zones/devices/sensors`, rejects malformed yaml
- **Do not:** mutate state from registry; registry is read-only
- **Commit:** Phase 10B.2: room registry loader

Slice 10B.3 — Adapter Interface

- **Goal:** Define the typed adapter interface. No implementations yet.
- **Files:** `src/room/adapters/contract.ts`, `tests/room/adapters/contract.test.ts`
- **Tests:** contract shape stable, every required method present
- **Do not:** implement fake or real adapter yet
- **Commit:** Phase 10B.3: adapter contract

Slice 10B.4 — Fake Room Adapter

- **Goal:** Reference implementation against the adapter contract.
- **Files:** `src/room/adapters/fake-room-adapter.ts`, `tests/room/adapters/fake-room-adapter.test.ts`
- **Tests:** implements every method; deterministic state; deterministic transitions
- **Do not:** introduce failure modes yet (next slice)
- **Commit:** Phase 10B.4: fake room adapter base

Slice 10B.5 — Fake Hue Bridge Simulator

- **Goal:** Fake bridge with `light/group` endpoints mirroring Hue v2 API shape.
- **Files:** `src/room/adapters/fake-hue-bridge.ts`, `tests/room/adapters/fake-hue.test.ts`
- **Tests:** GET state, PUT state, group ops; matches Hue v2 response shape
- **Do not:** simulate failure modes yet
- **Commit:** Phase 10B.5: fake Hue bridge

Slice 10B.6 — Fake Device Failure Modes

- **Goal:** Add offline, stale, timeout, auth-error, partial-success simulations.
- **Files:** `src/room/adapters/fake-failures.ts`, `tests/room/adapters/fake-failures.test.ts`
- **Tests:** each failure mode triggerable; clean state recovery
- **Do not:** persist failure state across restart (in-memory only this phase)
- **Commit:** Phase 10B.6: fake adapter failure modes

Slice 10B.7 — Adapter Conformance Suite

- **Goal:** The shared test suite every adapter must pass.
- **Files:** `tests/room/conformance/*.test.ts`
- **Tests:** read-state, command-on-off, command-dim, command-color, all failure modes, approval-flow, audit-trail, verification-read, rollback-compensation
- **Do not:** test against real Hue (no real adapter exists yet)
- **Commit:** Phase 10B.7: adapter conformance suite

Slice 10B.8 — Fake Device Event Emitter

- **Goal:** Fake adapter emits structured events into an in-memory bus (Phase 11 will persist).
- **Files:** `src/room/adapters/fake-events.ts`, `tests/room/events.test.ts`
- **Tests:** every fake action produces a typed event with full provenance
- **Do not:** persist events yet
- **Commit:** Phase 10B.8: fake adapter events

Slice 11A.1 — SQLite Event Store Scaffold

- **Goal:** Set up better-sqlite3, migrations runner, base schema.

- **Files:** `src/store/event-store.ts`, `db/migrations/0001_init.sql`, `tests/store/migrations.test.ts`
- **Tests:** migration runs; schema introspection matches expectation
- **Do not:** wire any projection yet
- **Commit:** Phase 11A.1: SQLite event store scaffold

Slice 11A.2 — Append-Only Property Test

- **Goal:** Prove the store cannot UPDATE or DELETE events.
- **Files:** `tests/store/append-only.test.ts`, `src/store/event-store.ts` (guard)
- **Tests:** attempts to update or delete fail; only INSERT and SELECT permitted
- **Do not:** add retention job yet (separate slice)
- **Commit:** Phase 11A.2: append-only invariant

Slice 11A.3 — Read-Only Projections

- **Goal:** Build the read-only projection layer used by Phase 12.
- **Files:** `src/store/projections/room-state.ts`, `src/store/projections/recent-traces.ts`, `tests/store/projections.test.ts`
- **Tests:** projections produce expected shape; cannot be coerced into writes
- **Do not:** wire UI yet
- **Commit:** Phase 11A.3: read-only projections

Slice 12A.1 — Tauri Shell Skeleton

- **Goal:** Tauri wraps the Next.js app, binds 127.0.0.1, opens window.
- **Files:** `src-auri/`, `src-auri/auri.conf.json`, `tests/auri/binding.test.ts`
- **Tests:** binds to 127.0.0.1 only; refuses to expose public port
- **Do not:** add IPC handlers yet
- **Commit:** Phase 12A.1: Tauri shell skeleton

Slice 12A.2 — /rest Orb Skeleton

- **Goal:** Three.js scene at `/rest` renders idle orb.
- **Files:** `src/app/rest/page.tsx`, `src/components/orb/`, `tests/orb/render.test.ts`
- **Tests:** scene renders; state transitions visible; no buttons exist in DOM
- **Do not:** add Working/Audit yet
- **Commit:** Phase 12A.2: rest orb skeleton

18. Move-In Readiness Plan

Before MacBook / Hardware

Item	Status
Fake room defined	Phase 10B
Fake devices implemented	Phase 10B
Adapter conformance suite	Phase 10B
Event store ready	Phase 11
Read-only projections	Phase 11
Command Center UI on synthetic data	Phase 12
Demo mode working	Phase 12
Bootstrap script	Phase 10A
Doctor command	Phase 10A
Onboarding documentation	Phase 20 (drafted earlier)
No real-hardware assumptions in any code path	All phases

When MacBook Arrives

1. `brew install node@22 pnpm rust ollama`
2. `git clone <repo>`
3. `pnpm bootstrap`
4. `pnpm doctor` → green
5. `pnpm test` → all tests pass
6. `pnpm dev` → Tauri opens, `/rest` orb renders
7. `pnpm demo` → demo mode active, synthetic room visible
8. `ollama pull llama3.2:3b`
9. Restart, verify model call lands in cost panel
10. Run fake room walkthrough
11. Verify Command Center fully functional with fake data
12. Verify no real hardware required for any flow

When Hue / Sensors Arrive

1. Add Hue bridge IP + API key to `config/room/hue.yaml`
2. Run `pnpm test:adapter:hue:read-only` → green
3. Restart, observe real bridge state in cockpit alongside fake devices
4. Run conformance suite against real adapter → 100% pass before next step
5. Enable Phase 16C dry-run on one light only
6. Verify dry-run plan visible in cockpit
7. Enable Phase 16D approval-gated on the same one light
8. Approve one command, verify execution + verification read + audit event
9. Expand to additional lights one at a time
10. Sensors follow same pattern (read-only forever)

When Camera Arrives (Deferred Beyond This Plan)

- Camera realization is a deferred sub-phase
- Requires explicit user action to enable
- Default: mock camera; real camera disabled
- Same fake-conformance-real path

19. Fortress Integration Summary

Phase 19 ships the four high-signal capabilities. Each is purely additive over Phases 10-18. Each is read-only or sandboxed. None introduce new authority surfaces.

Fortress Capability	Phase	Authority	Reads From	Effect
Architecture Graph + Runtime Dependency Visualizer	19A	Read-only	Static code analysis + telemetry	Interactive system map
Live Telemetry / Observability Cockpit	19B	Read-only	Event store via projections	Real-time visibility into cost, latency, errors
Governance / Safety Boundary Visualizer	19C	Read-only	Trust classes, approval gates, disabled matrix	Visible safety surface
Red-Team / Attack Simulation Layer (CAI)	19D	Sandboxed, approval-gated	Whitelist targets only	Adversarial testing of JARVIS itself

The fortress narrative for portfolio/recruiter use:

“JARVIS doesn’t just have governance — you can see it. The architecture graph shows every module and how it connects. The telemetry cockpit shows what’s actually happening in real time. The governance visualizer shows every safety boundary and turns red if any is violated. And the red-team layer lets JARVIS try to attack itself, proving that the boundaries hold under adversarial pressure. This isn’t a system that claims to be safe. It’s a system that demonstrates it.”

The fortress is the moment where a recruiter or technical interviewer goes from “impressive student project” to “you should be hired immediately.”

20. Final Recommendation

Recommended next phase: Phase 10 — Local Bootstrap & Room OS Substrate

Recommended first implementation target: Slice 10A.1 — Mac Bootstrap Contract

Recommended software to install first (on current machine): - better-sqlite3 (npm) - React Flow (npm) - Three.js + react-three-fiber (npm) - Playwright (npm)

Recommended fake-room target: A complete default-room.yaml with desk lamp, bed lamp, LED strip, smart plug, motion sensor, presence sensor — all fake-adapter-backed, all passing conformance suite, all visible in synthetic Command Center.

Recommended first real hardware adapter: Philips Hue light. Specifically: one bulb (e.g., Hue White Bulb) plus the Hue Bridge v2. Single light makes the three-stage gate (read-only → dry-run → approved) testable in isolation before expansion.

What must remain deferred: - Wake word, always-listening (deferred indefinitely) - Background camera (deferred beyond Phase 20) - Whole-home / multi-room (deferred beyond Phase 20) - Postgres, Redis, vLLM (deferred indefinitely) - Hue Cloud Remote API (deferred indefinitely) - Voice-only approval (deferred indefinitely) - Auto-approval of any class (deferred indefinitely) - Robotics, neural interfaces (out of scope for this plan)

Final verdict: safe with notes

The plan is safe because every operationalization slot is gated by existing governance contracts. The notes are:

1. **Adapter conformance is the load-bearing discipline.** If the fake-to-real conformance test ever becomes a formality, the whole architecture loses its safety properties. Treat conformance as non-negotiable.
2. **The fortress phase is high-leverage but high-risk.** CAI specifically introduces an adversarial capability. The sandbox boundary is the critical control. Treat Phase 19D testing as the most adversarial review of the project.
3. **The pace from Phase 9 to Phase 20 should be slower than Phase 1 to Phase 9.** The early phases were scaffold-heavy and intentionally cheap to ship. The operationalization era touches real bytes, real audio, real device commands. Each phase deserves the closeout discipline, not the velocity of the substrate era.
4. **MacBook arrival is the only timeline anchor that matters.** Everything before MacBook arrival is preparation. Everything after MacBook arrival should follow the move-in readiness checklist precisely. Don’t skip steps because the system is “working” — the checklist is what catches the silent failures.
5. **The fortress upgrades earn the portfolio outcome.** Phases 10-18 build the system. Phase 19 makes it visibly governed in a way no other student project will be. Phase 20 packages it. The combination is what turns “smart student” into “we need to hire this person.”